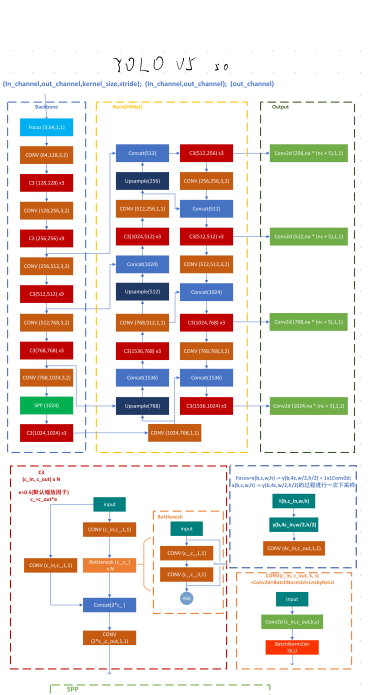
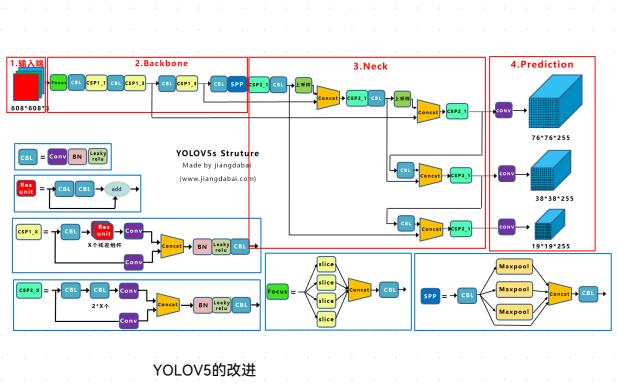
YOLO V5 s0


(in_channel,out_channel,kernel_size,stride); (in_channel,out_channel); (out_channel)
Backbone
Neck(PANet)
Output
Focus (3,64,1,1)
SPP (1024)
Bottleneck
SPP
(c_in, c_out)


1.输入端
2.Backbone
3.Neck
4.Prediction
608*608*3
YOLOV5s Struture
Made by jiangdabai
(www.jiangdabai.com)
76*76*255
38*38*255
19*19*255

输入端

①

Mosaic数据增强 同V4

自适应锚框计算
在 yolov3、v4 中是采用 kmean 和遗传算法对自定义数据集进行分析，获得合适自定义数据集中对象边界框预测的预设锚点框，计算初始锚框的值是通过单独的程序运行的。
在YOLO V5 中锚定框是基于训练数据自动学习的，此功能嵌入到代码中，每次训练时，自适应的计算不同训练集中的最佳锚框值。
在网络训练中，网络在初始锚框的基础上输出预测框，进而和真实框groundtruth进行比对，计算两者差距，再反向更新，迭代网络参数。
因此初始锚框也是比较重要的一部分

自适应图片缩放　　datasets.py的letterbox函数中
在项目实际使用时，很多图片的长宽比不同，因此缩放填充后，两端的黑边大小都不同，而如果填充的比较多，则存在信息冗余，影响推理速度。
Yolov5的代码中datasets.py的letterbox函数中进行了修改，对原始图像自适应的添加最少的黑边。
图像高度上两端的黑边变少了，在推理时，计算量也会减少，即目标检测速度会得到提升。
通过这种简单的改进，推理速度得到了37%的提升，可以说效果很明显。
训练时没有采用缩减黑边的方式，还是采用传统填充的方式，即缩放到416*416大小。只是在测试，使用模型推理时，才采用缩减黑边的方式，提高目标检测，推理的速度。

YOLOV5的改进
1、backbone：CSPDarkNet53+Focus
2、neck：SPP+PAN
3、head：YOLOv3
4、自适应图片缩放
5、数据增强：马赛克（Mosaic）
6、自适应锚框计算
7、激活函数：Leaky ReLU 和 Sigmoid 激活函数。
8、损失函数：GIOU
9、跨网格预测（新的Loss计算方法）

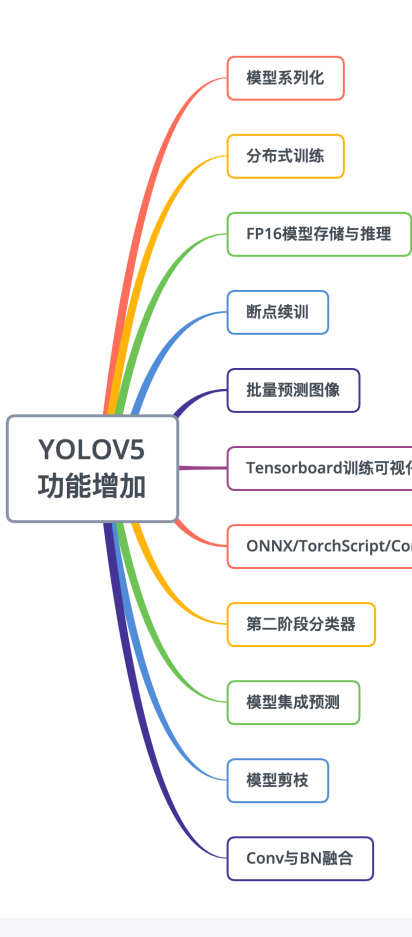
yolov5:

(1) 考虑了邻域的正样本anchor匹配策略，增加了正样本。

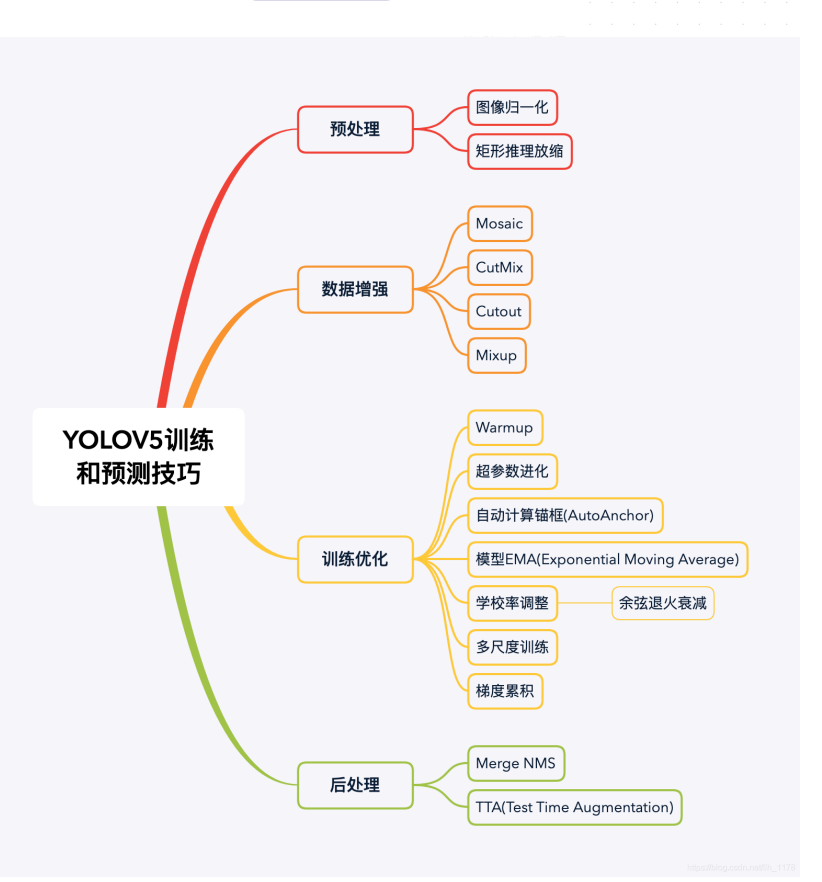
(2) 通过灵活的配置参数，可以得到不同复杂度的模型。

(3) 通过一些内置的超参优化策略，提升整体性能。

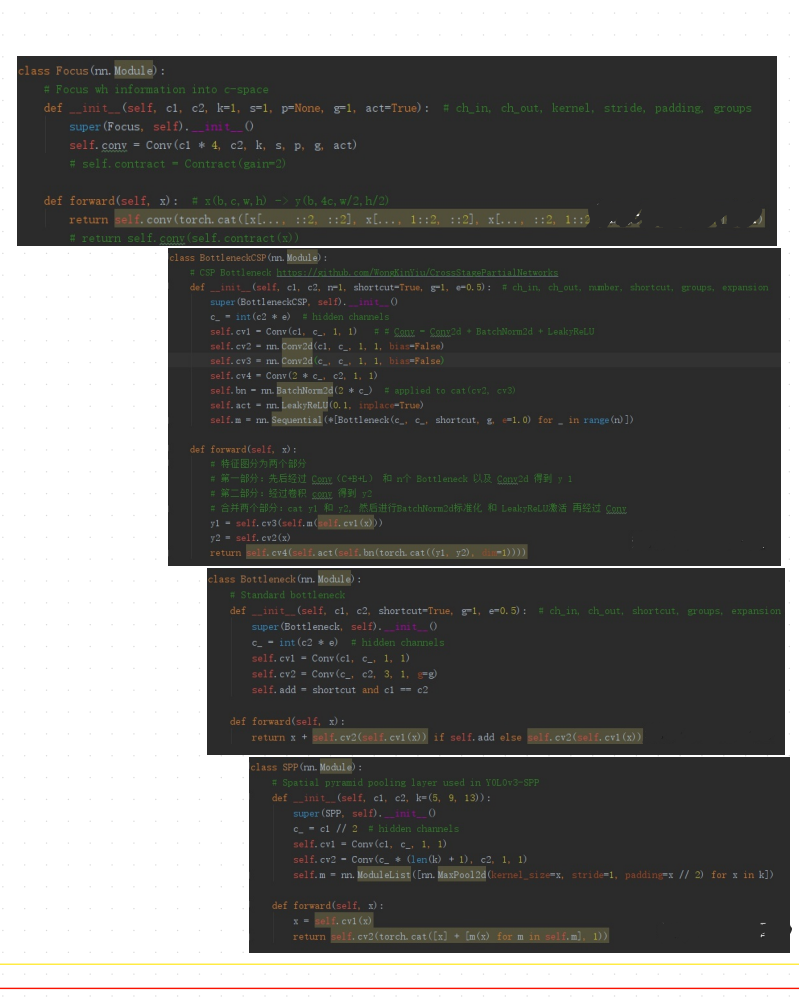
(4) 和yolov4一样，都用了mosaic增强，提升小物体检测性能。



YOLOV5功能增加
- 模型系列化
- 分布式训练
- FP16模型存储与推理
- 断点续训
- 批量预测图像
- Tensorboard可视化训练
- ONNX/TorchScript/CoreML输出
- 第二阶段分类器
- 模型集成预测
- 模型剪枝
- Conv和BN融合



YOLOV5训练和预测技巧
- 预处理
  - 图像归一化
  - 矩形推理图像
- 数据增强
  - Mosaic
  - CutMix
  - Cutout
  - Mixup
- 训练优化
  - Warmup
  - 超参数进化
  - 自动计算锚框(AutoAnchor)
  - 模型EMA(Exponential Moving Average)
  - 学习率调整
    - 余弦退火衰减
  - 多尺度训练
  - 梯度累积
- 后处理
  - Merge NMS
  - TTA(Test Time Augmentation)



```
class Focus(nn.Module):
    # Focus wh information into c-space
    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, act=True):  # ch_in, ch_out, kernel, stride, padding, groups
        super(Focus, self).__init__()
        self.conv = Conv(c1 * 4, c2, k, s, p, g, act)
        # self.contract = Contract(gain=2)

    def forward(self, x):  # x(b,c,w,h) -> y(b,4c,w/2,h/2)
        return self.conv(torch.cat([x[..., ::2, ::2], x[..., 1::2, ::2], x[..., ::2, 1::
        # return self.conv(self.contract(x))
```

```
class BottleneckCSP(nn.Module):
    # CSP Bottleneck https://github.com/WongKinYiu/CrossStagePartialNetworks
    def __init__(self, c1, c2, n=1, shortcut=True, g=1, e=0.5):  # ch_in, ch_out, number, shortcut, groups, expansion
        super(BottleneckCSP, self).__init__()
        c_ = int(c2 * e)  # hidden channels
        self.cv1 = Conv(c1, c_, 1, 1)    # # Conv = Conv2d + BatchNorm2d + LeakyReLU
        self.cv2 = nn.Conv2d(c1, c_, 1, 1, bias=False)
        self.cv3 = nn.Conv2d(c_, c_, 1, 1, bias=False)
        self.cv4 = Conv(2 * c_, c2, 1, 1)
        self.bn = nn.BatchNorm2d(2 * c_)  # applied to cat(cv2, cv3)
        self.act = nn.LeakyReLU(0.1, inplace=True)
        self.m = nn.Sequential(*[Bottleneck(c_, c_, shortcut, g, e=1.0) for _ in range(n)])

    def forward(self, x):
        # 特征图分为两个部分
        # 第一部分：先后经过 Conv (C+B+L) 和 n个 Bottleneck 以及 Conv2d 得到 y 1
        # 第二部分：经过卷积 conv 得到 y2
        # 合并两个部分：cat y1 和 y2，然后进行BatchNorm2d标准化 和 LeakyReLU激活 再经过 Conv
        y1 = self.cv3(self.m(self.cv1(x)))
        y2 = self.cv2(x)
        return self.cv4(self.act(self.bn(torch.cat((y1, y2), dim=1))))
```

```
class Bottleneck(nn.Module):
    # Standard bottleneck
    def __init__(self, c1, c2, shortcut=True, g=1, e=0.5):  # ch_in, ch_out, shortcut, groups, expansion
        super(Bottleneck, self).__init__()
        c_ = int(c2 * e)  # hidden channels
        self.cv1 = Conv(c1, c_, 1, 1)
        self.cv2 = Conv(c_, c2, 3, 1, g=g)
        self.add = shortcut and c1 == c2

    def forward(self, x):
        return x + self.cv2(self.cv1(x)) if self.add else self.cv2(self.cv1(x))
```

```
class SPP(nn.Module):
    # Spatial pyramid pooling layer used in YOLOv3-SPP
    def __init__(self, c1, c2, k=(5, 9, 13)):
        super(SPP, self).__init__()
        c_ = c1 // 2  # hidden channels
        self.cv1 = Conv(c1, c_, 1, 1)
        self.cv2 = Conv(c_ * (len(k) + 1), c2, 1, 1)
        self.m = nn.ModuleList([nn.MaxPool2d(kernel_size=x, stride=1, padding=x // 2) for x in k])

    def forward(self, x):
        x = self.cv1(x)
        return self.cv2(torch.cat([x] + [m(x) for m in self.m], 1))
```

Focus

Focus从高分辨率图像中，周期性的抽出像素点重构到低分辨率图像中，即将图像相邻的四个位置进行堆叠，聚焦wh维度信息到c通道空间，提高每个点感受野，并减少原始信息的丢失，该模块的设计主要是减少计算量加快速度。

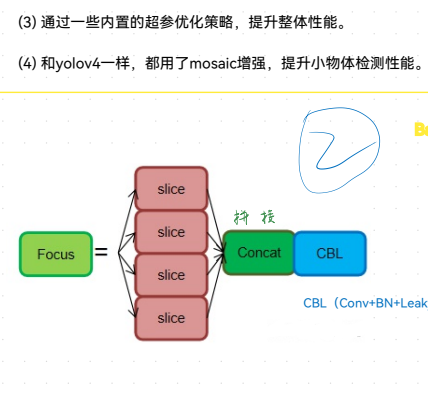
以Yolov5s的结构为例，原始640*640*3的图像输入Focus结构，采用切片操作，先变成320*320*12的特征图，再经过一次32个卷积核的卷积操作，最终变成320*320*32的特征图。

简单来说就是把数据切分为4份，每份数据都是相当于2倍下采样得到的，然后在channel维度进行拼接，最后进行卷积操作。

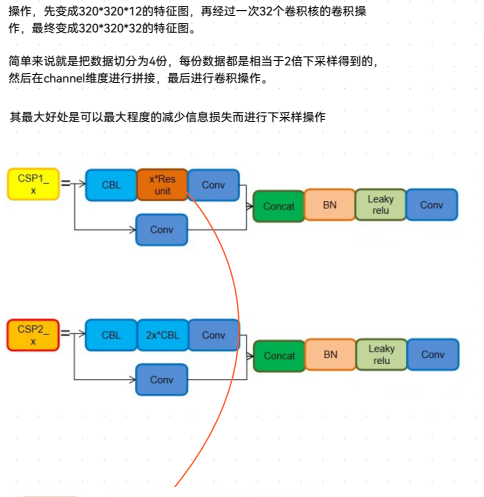
其最大好处是可以最大程度的减少信息损失而进行下采样操作

②

Backbone


Focus
slice
slice
slice
slice
拼接
Concat
CBL

CBL（Conv+BN+LeakyRelu)


CSP1_X
CBL
x*Res unit
Conv
Conv
Concat
BN
Leaky relu
Conv
CSP2_X
CBL
2x*CBL
Conv
Conv
Concat
BN
Leaky relu
Conv
Res unit
CBL
CBL
add

CSP（跨阶段局部网络）

跨阶段局部网络缓解以前需要大量推理计算的问题。

Yolov4中只有主干网络使用了CSP结构，而Yolov5中设计了两种CSP结构。

以Yolov5s网络为例，CSP1_X结构应用于Backbone主干网络，另一种CSP2_X结构则应用于Neck中。

Yolov4的Neck结构中，采用的都是普通的卷积操作。

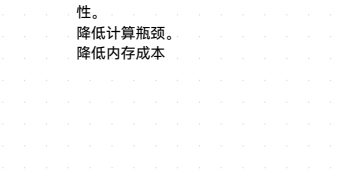
Yolov5的Neck结构中，采用借鉴CSPnet设计的CSP2结构，加强网络特征融合的能力。

增强了CNN的学习能力，能够在轻量化的同时保持准确性。
降低计算瓶颈。
降低内存成本

SPP

SPP模块（空间金字塔池化模块），分别采用5、9、13的最大池化，再进行concat融合，提高感受野。

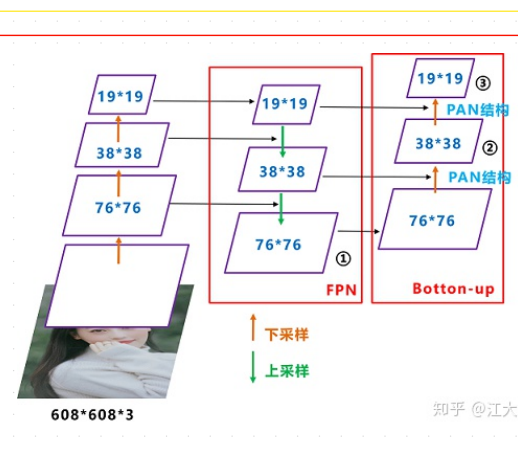
SPP的输入是512x20x20，经过1x1的卷积层后输出256x20x20，然后经过并列的三个Maxpool进行下采样，将结果与其初始特征相加，输出1024x20x20，最后用512的卷积核将其恢复到512x20x20。

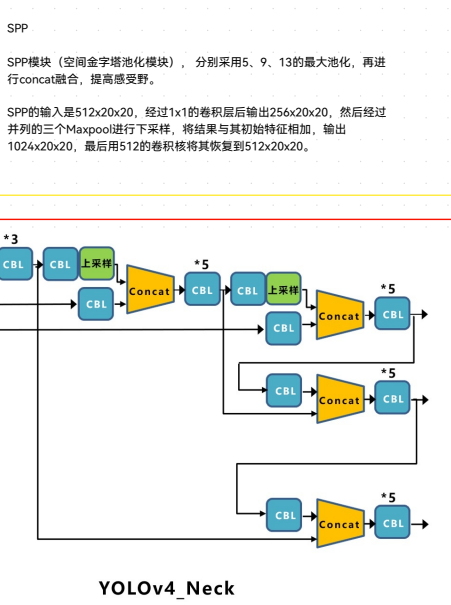

SPP
CBL
Maxpool
Maxpool
Maxpool
Concat
CBL

Neck

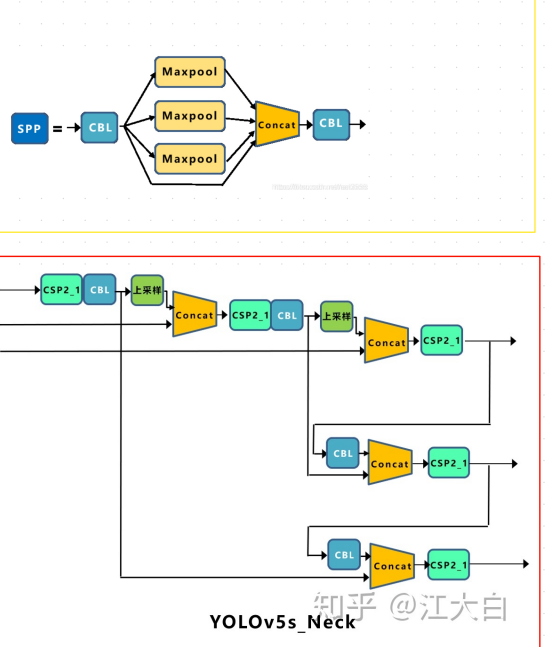
FPN+PAN

③


19*19
38*38
76*76
19*19
38*38
76*76
①
FPN
19*19
③
PAN结构
38*38
②
PAN结构
76*76
Botton-up
下采样
上采样
608*608*3
知乎 @江大白


*3
CBL
CBL
上采样
CBL
Concat
*5
CBL
CBL
上采样
CBL
Concat
*5
CBL
CBL
Concat
*5
CBL
CBL
Concat
*5
CBL

YOLOv4_Neck


CSP2_1
CBL
上采样
Concat
CSP2_1
CBL
上采样
Concat
CSP2_1
CBL
Concat
CSP2_1
CBL
Concat
CSP2_1
知乎 @江大白

YOLOv5s_Neck

出端

④

Bounding box损失函数

GIOU　　Generalized Intersection over Union

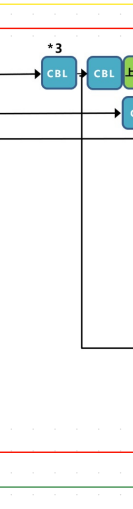
Yolov5采用GIOU_Loss做Bounding box的损失函数，使用 二进制交叉熵(BCE) 和 Logits 损失函数 计算类概率和目标得分的损失。

进化二：不相交时，IOU=0，两个框距离变换，IOU loss不变，改进为GIOU。

GIOU Loss，在IOU的基础上引入了预测框和真实框的最小外接矩形。

$$GIoU = IoU - \frac{|C-(A\cup B)|}{|C|}$$

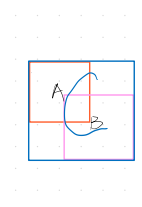
$$L_{GIoU} = 1 - GIoU$$



GIoU考虑到了 IoU 没有考虑到的非重叠区域，能够反应出 A,B 重叠的方式

当两框完全重合时取最小值0，当两框相切时，损失函数值为1；
当两框分离且距离很远时，损失函数值为2。
使用外接矩形的方法不仅可以反应重叠区域的面积，还可以计算非重叠区域的比例，因此GIOU损失函数能更好的反应真实框和预测框的重合程度和远近距离

激活函数
YOLO V5的作者使用了 Leaky ReLU 和 Sigmoid 激活函数。
在 YOLO V5中，中间/隐藏层使用了 Leaky ReLU 激活函数，最后的检测层使用了 Sigmoid 形激活函数。
而YOLO V4使用Mish激活函数


target grid
xy range
target center

跨网格预测（新的Loss计算方法）
yolov5的loss设计和前面yolo系列差别比较大的地方就是正样本anchor区域计算。
(1) 对于任何一个输出层，抛弃了基于max iou匹配的规则，而是直接采用shape规则匹配，也就是该bbox和当前层的anchor计算宽高比，如果宽高比例大于设定阈值，则说明该bbox和anchor匹配度不够，将该bbox过滤暂时丢掉，在该层预测中认为是背景
(2) 对于剩下的bbox，计算其落在哪个网格内，同时利用四舍五入规则，找出最近的两个网格，将这三个网格都认为是负责预测该bbox的，可以发现粗略估计正样本数相比前yolo系列，至少增加了三倍。

(1) 不同于yolov3和v4，其gt bbox可以跨层预测即有些bbox在多个预测层都算正样本
(2) 不同于yolov3和v4，其gt bbox的匹配数范围从3-9个，明显增加了很多正样本(3是因为多引入了两个邻居)
(3) 不同于yolov3和v4，有些gt bbox由于和anchor匹配度不高，而变成背景
这种特别暴力增加正样本做法还是存在很大弊端，虽然可以加速收敛，但是由于引入了很多低质量anchor，对最终结果还是有影响的。

nms非极大值抑制

Yolov4在DIOU_Loss的基础上采用DIOU_nms的方式，而Yolov5中采用加权nms的方式

Yolov5四种网络结构

Yolo v5一共有四个模型，分别为Yolov5s、Yolov5m、Yolov5l、Yolov5x。

Yolov5s网络最小，速度最少，AP精度也最低，如果检测的以大目标为主，追求速度，倒也是个不错的选择。

其他的三种网络，在此基础上，不断加深加宽网络，AP精度也不断提升，但速度的消耗也在不断增加。

而且四个文件的内容基本上都是一样的，只有最上方的depth_multiple和width_multiple两个参数不同。

(1) Yolov5s.yaml

```
depth_multiple: 0.33  # model depth multiple
width_multiple: 0.50  # layer channel multiple
```

(2) Yolov5m.yaml

```
depth_multiple: 0.67  # model depth multiple
width_multiple: 0.75  # layer channel multiple
```

(3) Yolov5l.yaml

```
depth_multiple: 1.0  # model depth multiple
width_multiple: 1.0  # layer channel multiple
```

(4) Yolov5x.yaml

```
depth_multiple: 1.33  # model depth multiple
width_multiple: 1.25  # layer channel multiple
```

四种结构就是通过上面的两个参数，来进行控制网络的深度和宽度。

其中depth_multiple控制网络的深度，width_multiple控制网络的宽度。

控制四种网络结构的核心代码是yolo.py中下面的代码，存在两个变量，n和gd

```
n = max(round(n * gd), 1) if n > 1 else n  # depth gain
```

深度：四种网络结构中每个CSP结构的深度都是不同的。

yolov5s，第一个CSP1中，使用了1个残差组件，是CSP1_1。

Yolov5m，在第一个CSP1中，使用了2个残差组件，是CSP1_2。

Yolov5l，在第一个CSP1中，使用了3个残差组件，是CSP1_3。

Yolov5x，在第一个CSP1中，使用了4个残差组件，是CSP1_4。

在yolov5的代码中，控制宽度的核心代码是yolo.py文件里面的这一行：

```
c2 = make_divisible(c2 * gw, 8) if c2 != no else c2
```

宽度：四种yolov5结构在不同阶段的卷积核的数量都是不一样的，因此也直接影响卷积后特征图的第三维度，即厚度。

Yolov5s，第一个Focus结构中，最后卷积操作使用了32个卷积核，因此经过Focus结构，特征图的大小变成304*304*32。

yolov5m，第一个Focus结构中，最后卷积操作使用了48个卷积核，因此经过Focus结构，特征图的大小变成304*304*48。

yolov5l，yolov5x也是同样的原理。

images
1×3×608×608
Slice
starts ⟨1⟩
ends ⟨1⟩
axes ⟨1⟩
steps ⟨1⟩
Concat
Focus
Conv
W ⟨32×12×3×3⟩
B ⟨32⟩
LeakyRelu
CSP
Add
BatchNormalization
MaxPool
Shape
Gather
Cast
Mul
Floor
Unsqueeze
Resize
Reshape
Transpose
1×3×76×76×85
482
1×3×38×38×85
463
1×3×19×19×85
output